UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Multimedia Communications

Adaptive Streaming

Marco Cagnazzo (marco.cagnazzo@unipd.it)
Department of Information Engineering (DEI)

Last classes

- Today: last lesson
- 22/05 lab session number 8
- 27/05 additional lab session + interviews for missed labs
- The report for the last labs (video codec) is due on June 12th

Video Communication through networks

- We know that a video source (uncompressed) emits bits at a *constant rate*
  - Example: uncompressed YCbCr HD video requires 1920×1080 pixels, with a typical frame rate of 50 Hz. Given 12 bits per pixel (4:2:0 sampling scheme), we end up with 1.244 Gbps
- After compression this rate is much reduced
  - Typical rates for this example are in the range $5 \div 20$ Mbps
- On the other hand, the coding rate is *no longer constant*
  - I frames are much "heavier" than P and B

Network performance metrics

- The compressed video stream generated at the encoder side is offered to a *Transport layer service*
- The Transport layer service offers an end-to-end channel for bitstreams
- However, the Transport layer does break the bitstream into smaller units (packets) that are individually sent to the destination
- The virtual channel provided by the Transport layer is characterized by:
  - Throughput
  - Delay
  - Jitter
  - Bandwidth-delay product
  - Bit-error probability
  - Packet loss ratio

Throughput

- Let us consider the connection between two entities (i.e., processes) at the application layer on two remote hosts
- The two peers see an end-to-end virtual channel between them, but actually, this is the service provided by the underlying transport layer

- Let $b(t)$ be the cumulative number of bits transmitted from the initial instant until instant t, seen at application layer

- The application layer **throughput** $s(t,T)$ is the average number of bits that has been transferred to the destination entity in the interval $(t-T,t)$, divided by the interval duration:

$$s(t;T) = \frac{b(t) - b(t-T)}{T}$$

Throughput

- This is more precisely referred to as *average throughput*
  - If we let $T \to 0$ in $s(t;T) = \frac{b(t)-b(t-T)}{T}$, we obtain the **instantaneous throughput** $s(t)$

$$s(t) = \frac{d}{dt} b(t)$$

  - It can also be interesting to consider so-called long-term throughput S, which sometimes is also referred to as **goodput:**

$$S = \lim_{\substack{t\to+\infty \\ T\to+\infty}} s(t,T)$$

It is the average throughput on an infinite-length interval

Throughput

- The instantaneous throughput offered by a Transport service typically varies over time
- There are multiple possible causes, but mainly related to
  - Bit errors during transmission, which requires retransmissions
  - Network congestion: too many packet are injected in a network, and the buffers of the routers undergo overflows
- The average throughput is typically rather stable
- The long-term throughput is by definition time independent; on the other hand, its mathematical definition cannot be applied as it is in practice
- However, **the average throughput computed on a reasonably long interval** is a good approximation of the long-term throughput: this is what we will consider in the next slides as throughput S

Delay

- We consider the end-to-end delay or **latency** as $d = t_{\text{dest}} - t_{\text{source}}$
- t_{source} is the instant when a data unit, or a **packet**, (e.g., an image or part of an image within a video sequence) is fully available at the transmitter side
- t_{dest} is the instant when the same data unit is fully available at the destination side
- The end-to-end delay can be decomposed as the sum of the **nodal delays**, i.e., the delays d_i experienced at every single hop of the path between source and destination

$$d = \sum_{i=1}^{N} d_i$$

Here N is the number of hops between source and destination

Delay

- The **nodal delay** d_i is given by four contributions:

$$d_i = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

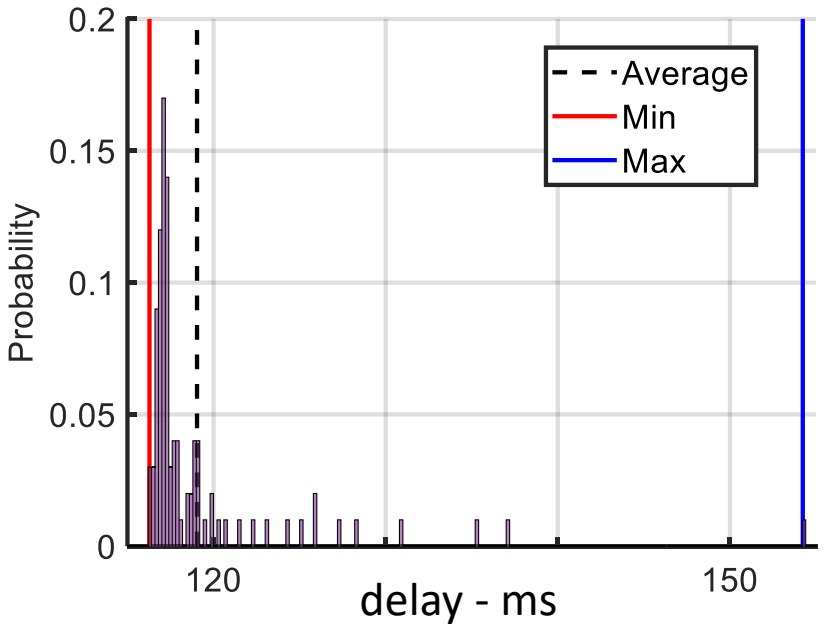
- d_{proc} = processing delay
- d_{queue} = queuing delay
- d_{trans} = transmission delay
- d_{prop} = propagation delay

Nodal delay

- The **processing** delay is the time needed by a router to process the packet (error control and routing, i.e. deciding in which output interface buffer the packet must be copied)
- The **queueing** time is the time spent in the output interface buffer. The packet must wait for other possible packets that need to use the same link. This delay therefore depends on the global number of packets that circulate in the network
- The **transmission** delay is the **signal duration**. E.g., if the link bit-rate is 1 Mbps, the duration of a signal representing one bit is $1\mu s$
  - In formula, $d_{trans} = L/R_C$ where L is the size of the data unit at PHY level and R_C is the link capacity (i.e., the throughput of the physical layer)
- The **propagation** delay is the time needed by the signal to travel the distance ℓ between the hosts at speed c: $d_{prop} = \ell/c$
- Transmission and propagation delay are two very different things!

Jitter

- The per-packet delay observed in one connection is typically very variable
- Since its value is determined by many different phenomena, it is convenient to model it as a **random process**
- Therefore, we are interested to the average delay
- But also the **variance** of the delay is important: it is called **jitter**



The jitter tells us if the delay is often close to its average value or not
This has an important impact on streaming services, as we will see later on

Reliability metrics: Packet Error Rate and Bit Error Rate

- PHY layer can deliver packets with errors
  - Low signal-to-noise ratio at the receiver, or strong interference → wrong bit decoding
  - Packets with (unrecoverable) errors are usually discarded by the receiver
- Packet Error Rate (PER):
  - fraction of packets with errors over the total number of transmitted packets
  - probability that a packet transmissions is affected by at least one bit error
- Bit Error Rate (BER)
  - It is the fraction of **bits** whose value at destination is not the same it was at the transmitter

BER and PER

- What is the PER of a packet of size L if the BER is ε?
- A packet is correctly received if and only if all its L bits are error-free
- The probability that each bit is correct is $1 - \varepsilon$

- Assuming independent errors,
- $Pr(packet\ received) = (1 - \varepsilon)^L$
- $PER = 1 - (1 - \varepsilon)^L$

Error detection and error corretion

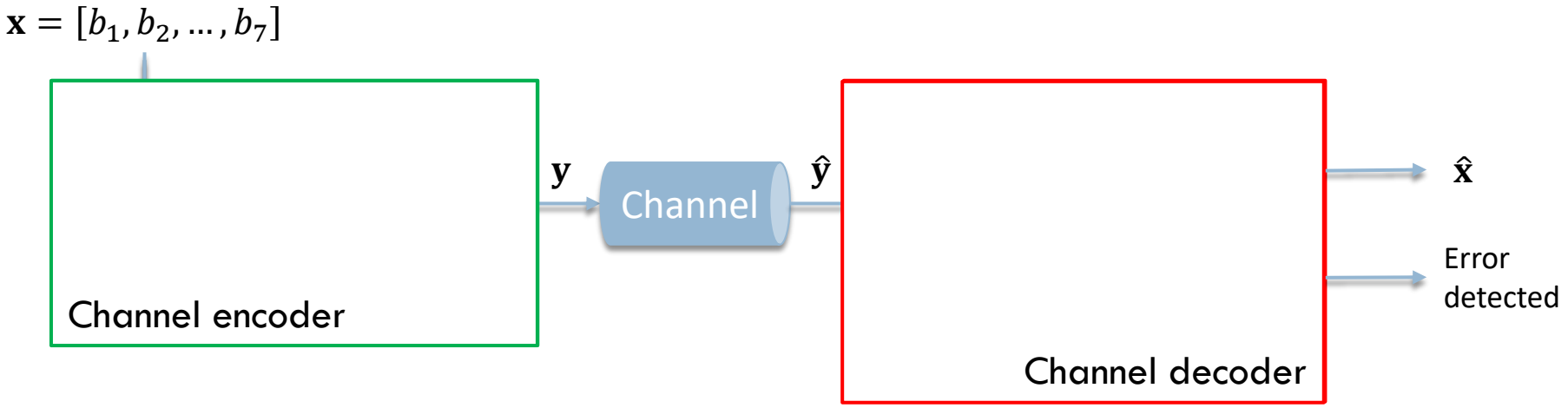
- All transmission links have some non zero bit error rate
- However, errors can be **detected** and **corrected**
- An error detection (ED) technique allows to recognize whether a packet is affected by bit errors
- An error correction (EC) technique allows to detect errors and to correct them
- Both ED and EC are based on the idea of **adding** redundance bits to protect the information
- Let k be the number of bits to be protected. After adding the protection bits, we have $n > k$ bits. The **coding ratio** is k/n
- ED and EC are referred to as *Channel Coding* in opposition to *Source Coding* (i.e., compression)

Example: Parity check

- A simple example of channel coding is *parity check*
- Data is structured into blocks of $k = 7$ bits:

$$\mathbf{x} = [b_1, b_2, \dots, b_7]$$

- A parity bit is computed: $b_8 = \sum_{i=1}^{7} b_7$ (modulo 2 sum)
- The encoded block is $\mathbf{y} = [\mathbf{x}\ \mathrm{b}_8] = [b_1, b_2, \dots, b_7, b_8]$



- Error detection over $\hat{\mathbf{y}}$: if $\sum_{i=1}^{8} \hat{b}_i \neq 0$, **at least one bit is wrong**

Channel coding trade-offs

- **Bit-rate**: Increasing $\frac{k}{n}$ reduces the overhead but decreases the ED and EC capability
- **Complexity**: more complex techniques (turbo-codes, LDPC, RS) achieve higher ED and EC for the same $\frac{k}{n}$ but request more computational power/more memory
- **Delay**: interleaving improves robustness to burst errors but introduces additional delay; convolutional codes also require some additional latency

Marco Cagnazzo

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

CODING RATE VS THROUGHPUT

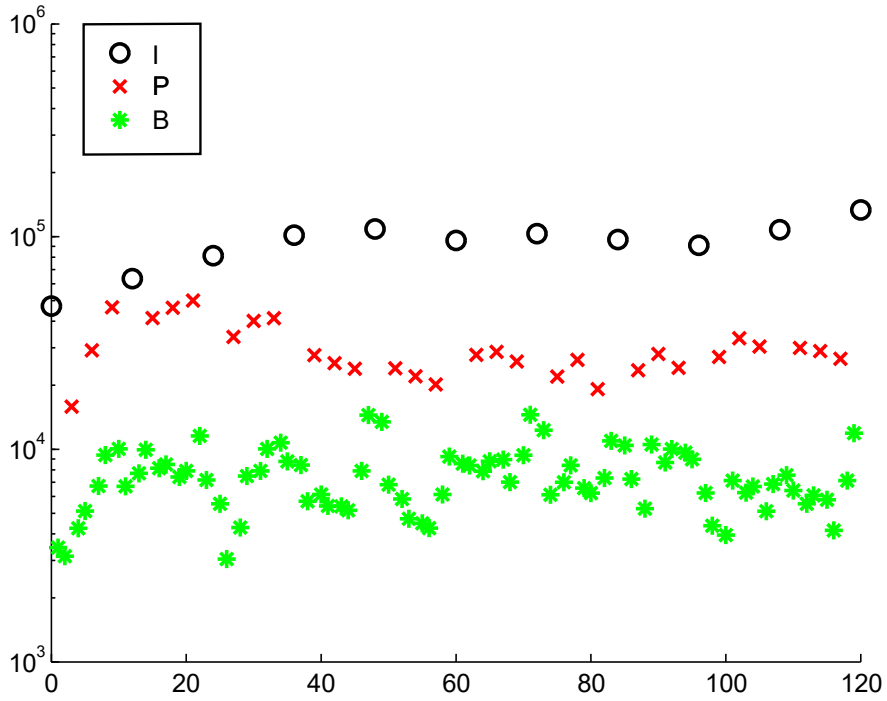
Setting the coding rate in video communications

- How to decide the coding rate R_C in video compression?
- We know that the QoE increases with R_C
  - In all experiments, subjective (MOS) and objective quality metrics (PSNR, SSIM, VMAF, …) increase with R_C for a given codec
- So, we should set as the coding rate the *largest possible* value
- **How large?**
- In video transmission over a network, the limit should be the long-term throughput at application level: $\boldsymbol{R_C \leq S}$

We can control the compression rate…

- Is it possible to configure a video encoder such that a given coding rate is achieved?

- Yes, by using *rate control techniques*
  - Rate control techniques **dynamically adjust the quantization step** of the video codec pursuing a given target rate
  - There is a large amount of research on rate control, and it is nowadays possible to control the rate of a video codec quite accurately
  - Yet, **the video coding rate is rarely constant at the frame level**
    - Intra frames require much more bits than P and B frames
  - **The bitrate can be considered roughly constant at the GoP level**



Frame size [bits] in a rate-controlled H.264 encoder

… but what about the throughput?

- Remember $R_C \leq S$

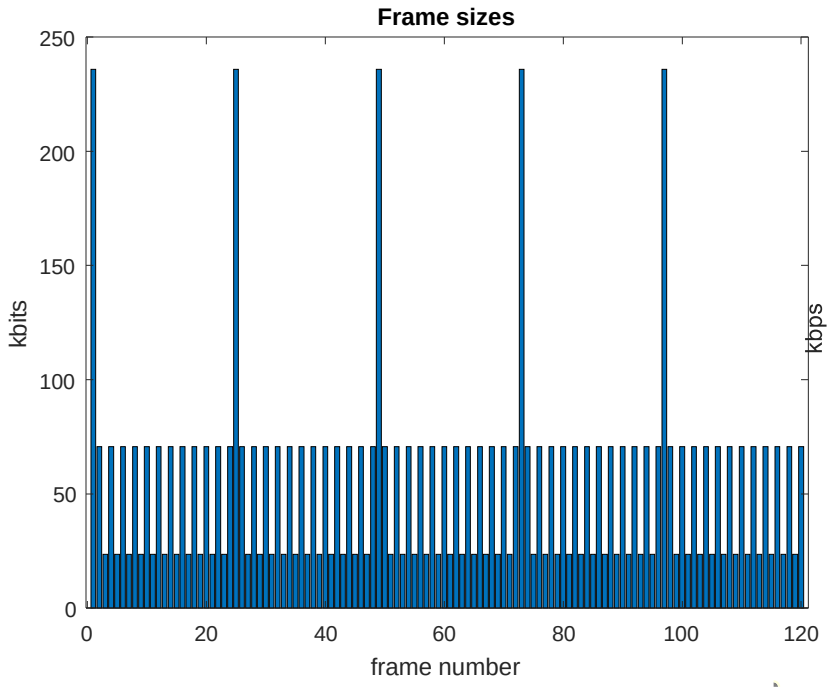
- Good news, we can control R_C; bad news, **we cannot control** S
  - The throughput is not under the control of video services providers
  - The throughput is often **not known at the encoding time**
    - In **streaming** applications, the encoding and transmission of a video can happen at different times
    - Exceptions: live streaming, where the content is generated at the time of transmission; and real-time communications. In these cases, S can be **estimated**
  - The throughput can change very quickly during the transmission, in particular for mobile users
  - All in all, in one-to-many services (broadcast), each user can experience a different throughput

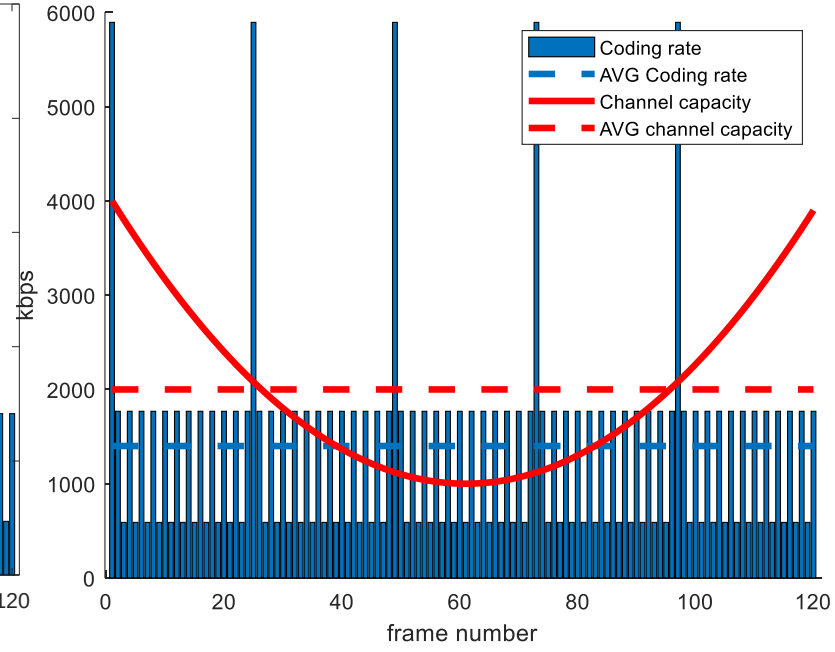
Why should we keep $R_C \leq S$?

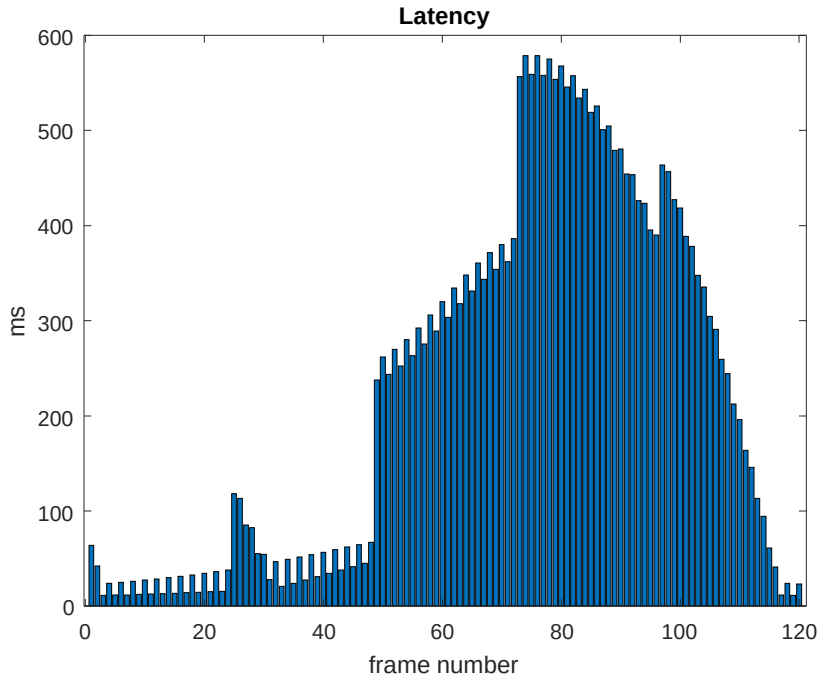
- Let us use the delay model to evaluate the latency experienced in video transmission
- We consider for simplicity a one-hop transmission
- The processing and propagation times are considered as constants
- Thus $d_{pkt} = \frac{L}{S} + T$
- d_{pkt} is the delay experienced by a packet. Let us consider one image per packet
- L is the compressed image (packet) size
- S is the throughput
- T is the sum of queuing, processing and propagation times
- The queueing times **accounts for previous packets waiting for being sent**

Example: variable-capacity channel

- Let us consider a IBPBP… GOP structure, and the transmission over a channel with variable throughput
- The average coding rate and throughput are: $R_C = 1.4$ Mbps and $S = \mathbf{2\ Mbps}$

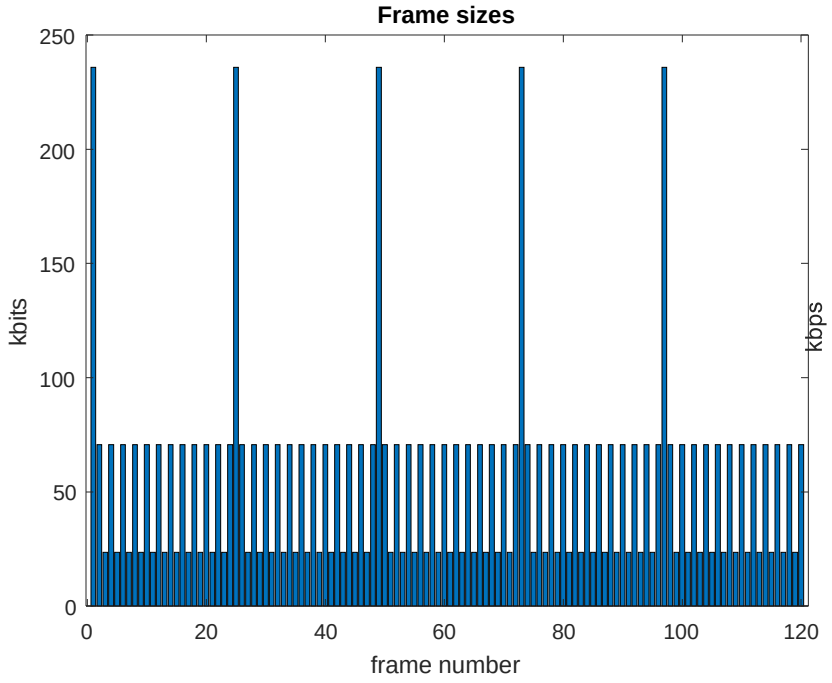


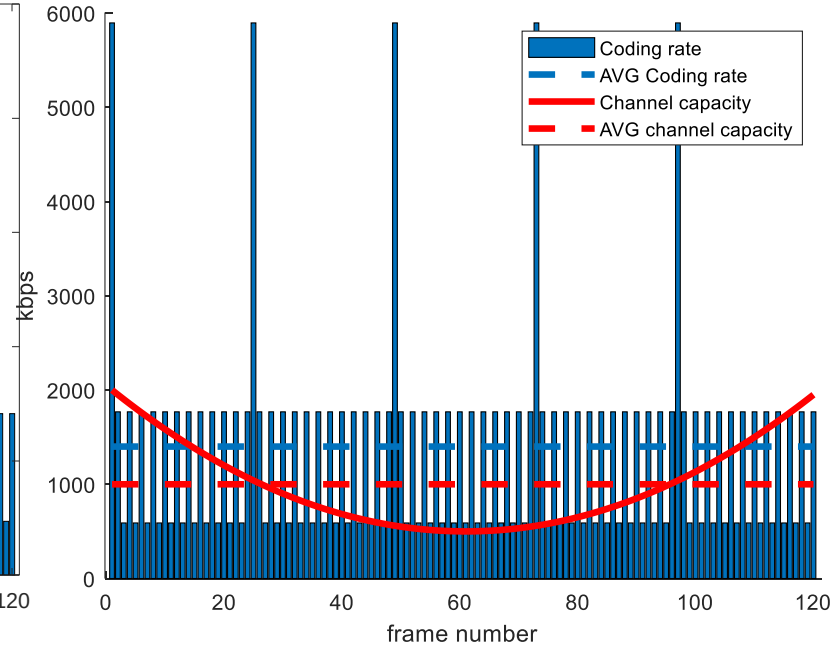


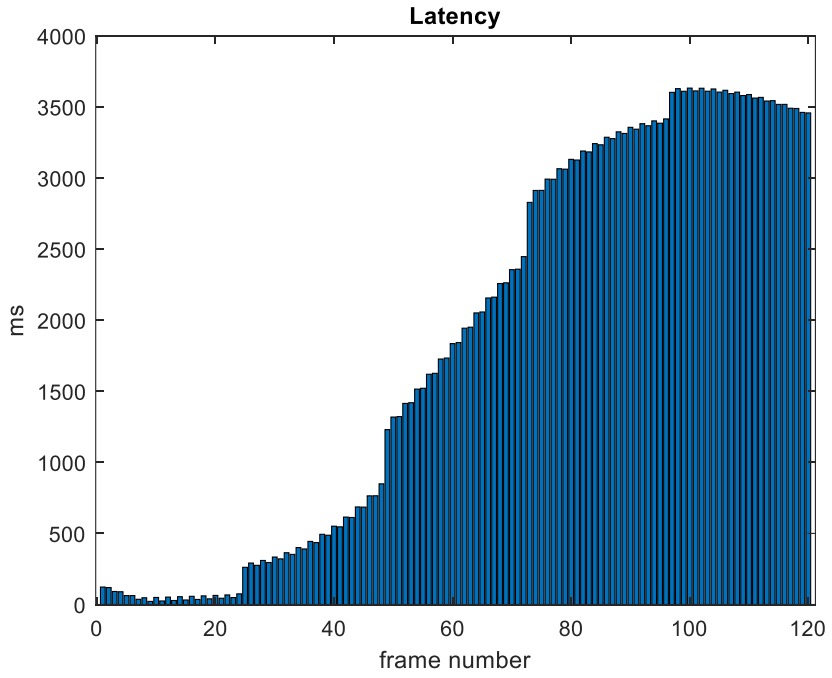


Example: variable-capacity channel

- Let us consider a IBPBP… GOP structure, and the transmission over a channel with variable throughput
- The average coding rate and throughput are: $R_C = 1.4$ Mbps and $S = \mathbf{1\ Mbps}$







Throughput and coding rate

- When $R_C > S$, the delay builds up
- If in a later moment the throughput increases, the delay can be adsorbed
- But we cannot be sure about that
- In the worst case, the output queue at the router fills, and the router starts discarding packets: a **congestion** is occurring
- A congestion typically tends to get worse: packet loss might imply retransmission which in turns makes the congestion worse
- In any case this disrupts any service and cannot be sustained in the long term

What if the throughput is not large enough?

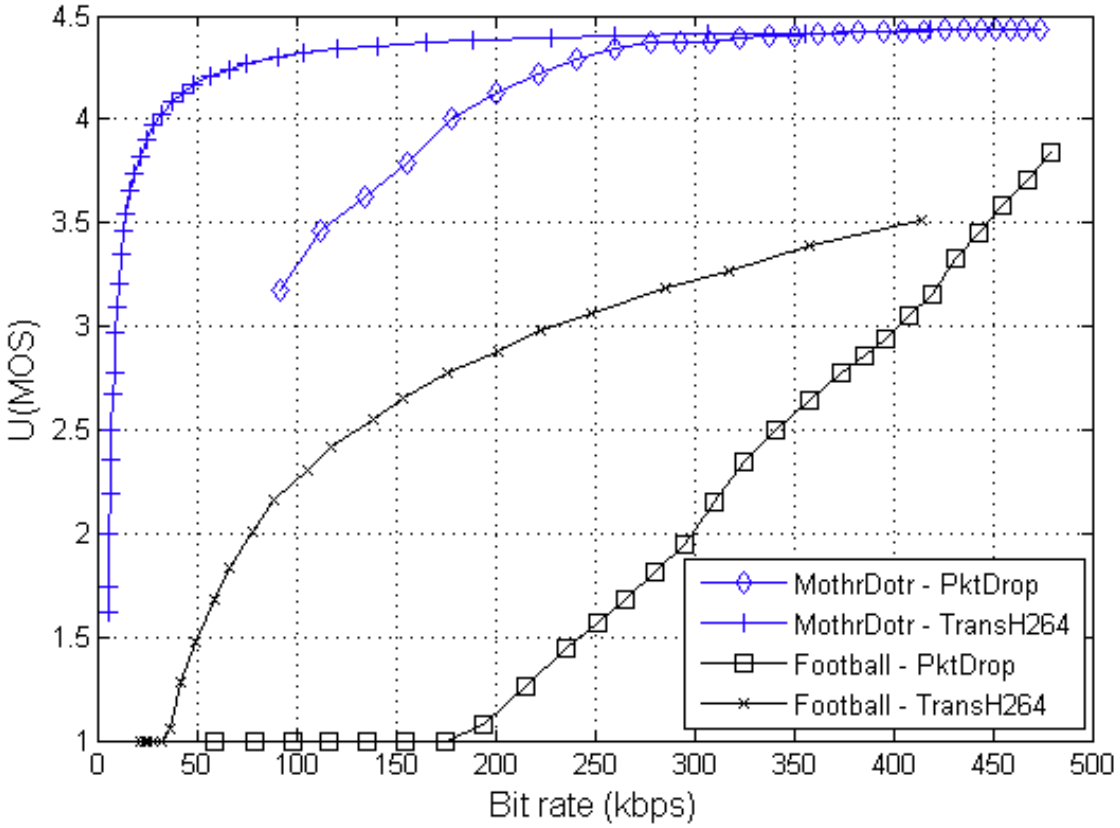
- We cannot control S, thus it is difficult to satisfy the constraint $R_C \leq S$ in all situations
  - Solution 0: use a very small coding rate. In this case the video quality will also be very small
- What happens if, in a link the throughput is $S < R_C$?
  - Sol. 1: the router ***drops*** packets to match the link's throughput
  - Sol. 2: the router ***transcodes*** the video to a lower bit-rate (and lower quality) to match the throughput
  - Sol. 3: **Freeze** the video while waiting for packets
  - Sol. 4: **Scalability** (smart packet drop)
  - Sol. 5: **Adaptive video streaming** (client driven)

Packet drop vs. transcoding

- If the router sees that the queue on its output interface increases steadily, it means that $R_C > S$
- In this case, a very simple option is just **dropping** random packets until the queue falls below a given threshold
  - Advantage: it is an extremely simple method
  - Inconvenient: it potentially destroys video quality (uncontrolled quality loss)
- Second solution: **transcoding** (cross-layer solution)
  - The router decodes the video packets and encodes them again at lower quality/resolution so that the link throughput is matched
  - Note that transcoding can only be used to reduce the bitrate, but not to increase it
  - Advantage: a much better QoE (see next slide)
  - Inconvenient: it is much more complex, requires cross-layer devices, it has a high infrastructure cost

Packet drop vs. transcoding: MOS

Quality of dynamic video (Football) and a static video (Mother and Daughter) when applying **transcoding** (TransH264) and **packet dropping** (PktDrop)



Freeze events

- If $R_C > S$ and the routers do not perform transcoding nor packet drop, the player will experience a *stall event* (i.e., a freeze, or rebuffering event), which is very bad in terms of QoE
- **Let us illustrate how a stall event happens**
- We assume that the video is encoded in *segments* of T_S seconds
- Each segment contains an integer number of GoPs, such that **it can be decoded independently** from other segments
- We also assume that a segment **can only be played when it is fully received**
  - **This is a simplifying assumption, but it is close to what is done in many practical case**
- Thus, the client continuously asks for new segments
- We analyze what happens if the routers do not drop packets
  - E.g., because the queue is large enough to accommodate packets

System model

- Problem parameters:
  - T_S segment duration in seconds
  - R_C coding rate of the segment, in bits per second
  - S current throughput, in bits per second
  - T_D time needed to download a segment, in seconds
- **Remark: we will first consider that these parameters do not vary with time, but in a more realistic model at least *S* varies with time**
- Question: how many bits is a segment?
  - $B_S = R_C T_S$
- Question: how long does it takes to download a segment?
  - $T_D = \frac{B_S}{S} = \frac{R_C}{S} T_S$
- Thus, $T_D < T_S \Leftrightarrow R_C < S$, i.e., the time needed to download a segment is less than the segment duration if and only if the coding rate is less than the throughput
- *If this does not happen, we have a **freezing event***

Freeze the video

- Let us consider an example with:

$R_C = 500\ kbps$
$S = 400\ kbps$
$T_S = 1\ s$

We want to find the frequency and the duration of the freezing events (*stalls*)

Let us first find out the downloading time T_D: start with B_S

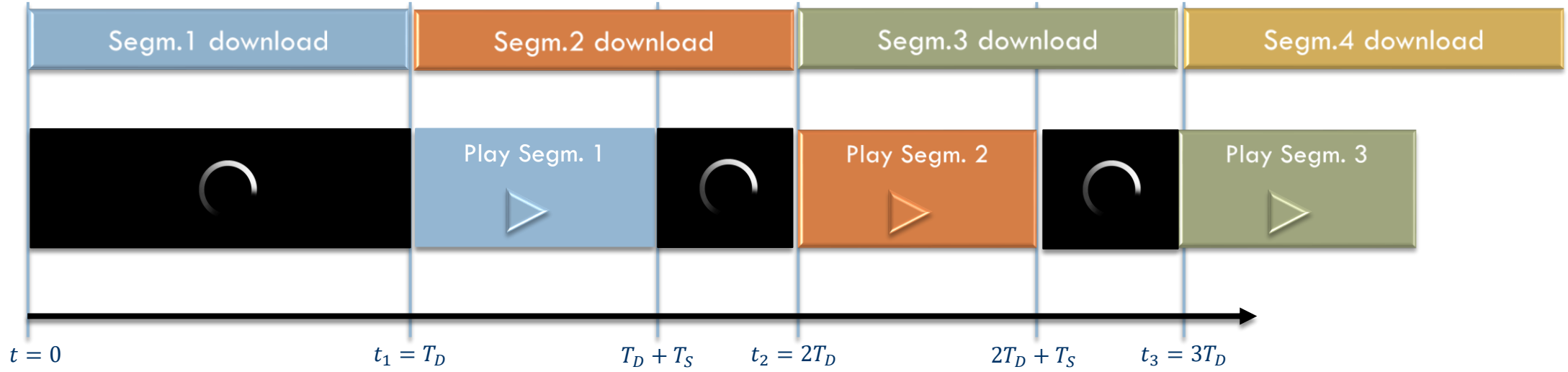
- $B_S = R_C T_S = 500$ kbits
- $T_D = \frac{B_S}{S} = \frac{R_C}{S} T_S = \frac{500\ \text{kbits}}{400\ \text{kbps}} = \mathbf{1.25s} > T_S$

  We need 1.25 s to receive one second (one segment) of video

- Remember that each segment is played only when it is fully received

Freeze the video

- Let us set $t = 0$ when the first bit of the first segment is received
- Thus, the first segment is fully received at $t_1 = T_D = 1.25$
- It is played between T_D and $T_D + T_S = 2.25$
- But at that time the second segment is not arrived yet! It will only arrive at $t_2 = 2T_D = 2.5 > T_D + T_S$
- In the meanwhile, the video **will freeze**
- Question: find the time t_n when the n-th segment is ready to be played and determine the frequency and duration of the freeze events



Freeze the video

- Question: find the time t_n when the n-th segment is ready to be played and determine the frequency and duration of the freeze events

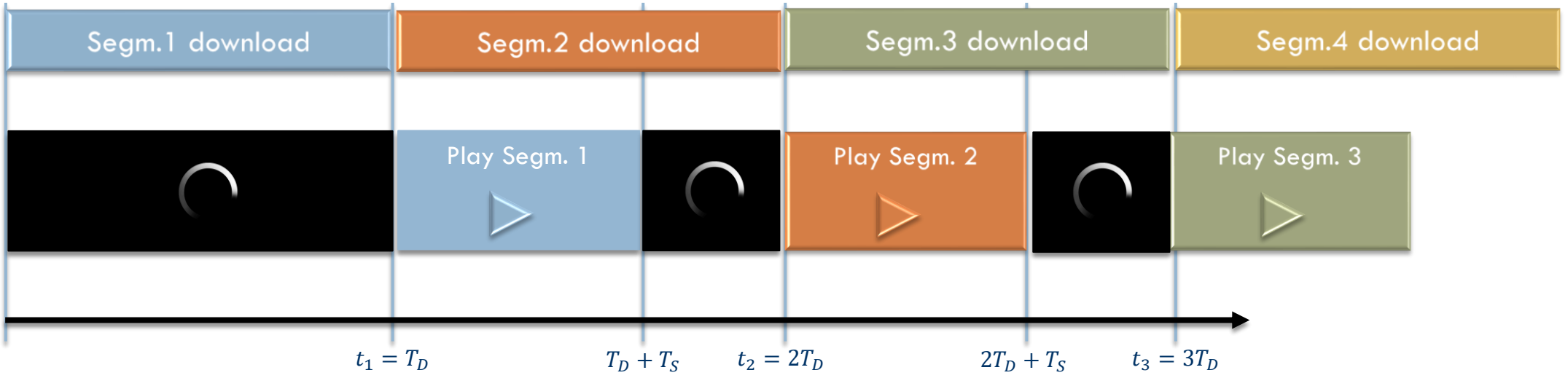
$t_n = nT_D$

The n-th segment is played between nT_D and $nT_D + T_S$

Thus, the freeze duration is $\Delta_n = (n+1)T_D - (nT_D + T_S) = \boldsymbol{T_D - T_S} = \left(\frac{R_C}{S} - 1\right) T_S$

We have a stall event every T_D seconds, thus $f_{\text{Stall}} = \frac{1}{T_D} = \frac{S}{R_C T_S}$

Long segments→long but less frequent stall; stall can only be avoided if $R_C \leq S$



Scalable video coding

- It is in theory a good solution
- It means that the video is already organized in quality layers (ie., a scalable codec is used)

- Intermediate routers, instead of dropping random packets, only drop those that less impair video quality
  - E.g., only B-frames are dropped
  - Or, only *refinement* information is dropped
  - Similar behavior as transcoding

- Nevertheless, scalability has *costs*
  - More complicated codecs
  - Less effective in R-D
  - Still requires updating routers (much less than transcoding but non-negligible), encoders and decoders

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

ADAPTIVE STREAMING

Adaptive video streaming

- Adaptive video streaming is currently the most widespread solution to the coding-rate-vs-throughput problem
- The system model is the following:

- We take into account **time-variable throughput**
- Just as before, the full video sequence is divided into **segments**
  - Typical duration of a segment: 1 to 10 seconds
- Again, each of the N segments is encoded independently from the others and thus is independently **decodable**

BUT …

Adaptive video streaming

BUT

- Each segment n with $n \in \{1,2,\dots,N\}$, **is encoded several times**
- Each of the different encodings of the segment is a **level**
- There exist K levels
- Each level $k \in \{1,2,\dots,K\}$ is characterized by a given **coding rate** $R_C(k)$ *that is independent from* n
  - This coding rate is achieved by possibly modifying the resolution, the frame rate, and the quality (quantization) of the segment
- **In other words: the video is encoded in N segments of fixed duration (e.g., 1 second) and each version is available at K different bit-rates, e.g., 200 kbps, 500kbps, 1Mbps, 2Mbps, 5Mbps, 10Mbps, 25 Mbps**

Adaptive video streaming

- The server makes available a *manifest file* with
  - The number of segments N
  - The duration of segments T_S
  - The number of levels K
  - The bit-rate of each level: $R_C(k),\ \forall k \in \{1,2,\dots,K\}$

Adaptive video streaming

- When a client requests a video, it first download the manifest file, and then uses it to request the individual segments
- The request is: please send me segment n at level $k = q(n)$
  - The request strategy $q(\cdot)$ is decided by the client
- The request is sent as soon as the previous segment $n - 1$ has been fully downloaded
- When a client plays a video, **it fetches the segments in sequence over HTTP**
  - Because HTTP requests/replies can go through most of the firewalls
- Therefore, *the client can decide the bit rate of each segment*
- This flexibility should be used to maximize the QoE:
  - Download the highest quality, but try to avoid stalling and stabilize quality
  - Research problem: what is the best strategy for the request strategy $q(\cdot)$?

Adaptive video streaming

- The request strategy is decided using an **adaptive bit rate (ABR) algorithm**
- To prevent video freezes, the client has a buffer that can store a number L of segments
- It is called *playout buffer*
- ABR is used to decide the bitrate of each downloaded segment according to the current estimated throughput and/or the playout buffer status
  - The client tunes down the bitrate when the network degrades and or when the playout buffer becomes too small
  - When the network improves it tunes the bitrate up to provide a better experience
  - Therefore, the client must be able to estimate the throughput: how? it is an open research field

- We indicate with $B(t)$ the amount of video **received and not yet played**
- $B(t)$ is measured in **seconds** of playable video, not in bits

Adaptive video streaming

- The client proactively fetches video segments from the server in advance and stores them in a buffer until they are ready to be played
- These segments are then played in the order they were received (First-In, First-Out)
- If the buffer becomes empty and no more segments are available for playback, the video will pause, causing a **rebuffering event**
- Different strategies can be used to decide when to start over the playout.
  - The playout can resume only when the next M segments are completely downloaded
  - M can be any integer value
  - The trade-off is between stall duration and stall frequency: the larger M, the longer we wait before resuming the playout, the less probable becomes the next stall

System model for AS

- Recap: for segment n we have:
  - T_S: playout duration of a video segment (constant)
  - S_n: throughput during the download of segment n
  - $q(n)$: requested level for segment n
  - $T_D(n,k)$: downloading time for segment n at level k

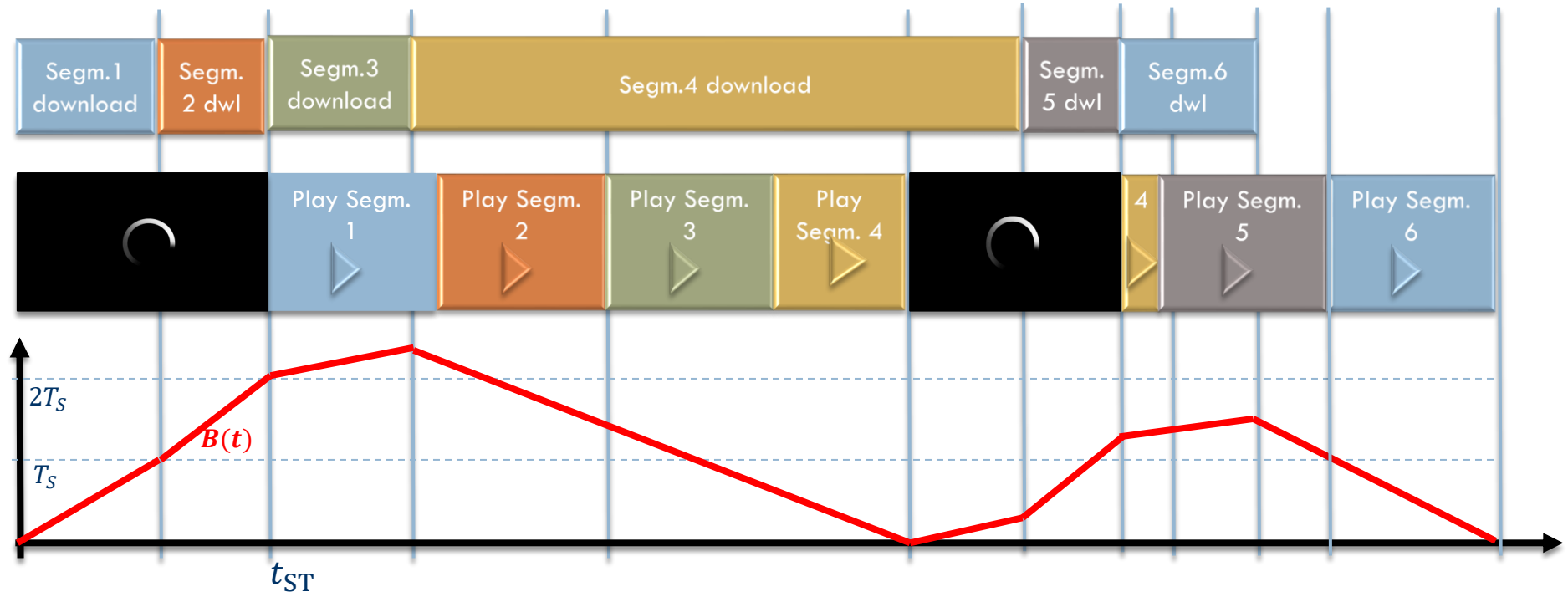
$$T_D(n,k) = \frac{T_S R_C(k)}{S_n}$$

We also have:

- $B(t)$: **the playout buffer**. At time t, $B(t)$ is the amount of video **received and not yet played.** It is measured in seconds
- The **initial buffering time** $T_{IB} = L \cdot T_S$: before starting the video playout, we fill the play-out buffer with L segments

The playout buffer

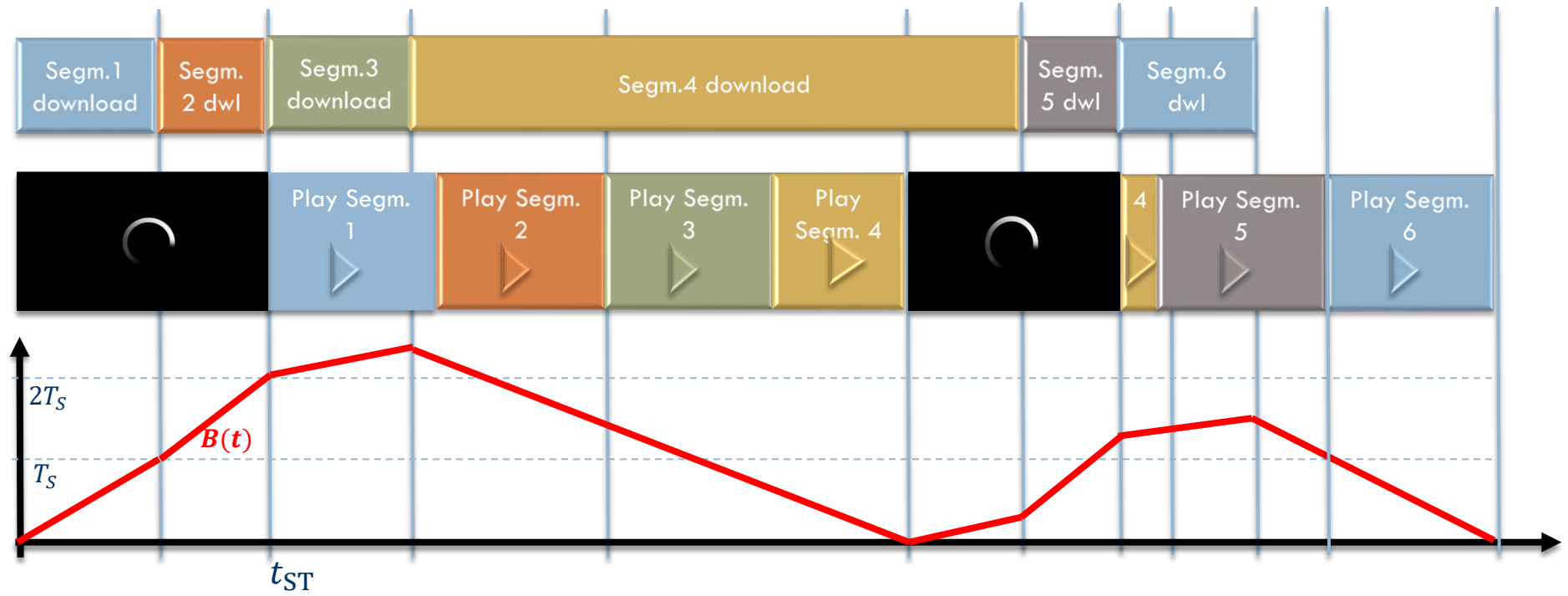
- The playout starts at t_{ST}, defined as the moment when the play-out buffer is filled with the first L segments of video
- When the buffer is void, the client wait until it receives M new segments
- Example for $L = 2, M = 1, N = 6$
- **Question: compute the slope of $B(t)$** - consider S constant during a segment dwl



The playout buffer

Compute the slope of $B(t)$ - consider S constant during a segment dwl

- During rebuffering of a segment, the size of the buffer increases of T_S in a time T_D
- $B'_{\text{rebuff}}(t) = \frac{T_S}{T_D} = \frac{T_S}{\frac{T_S R_C}{S}} = \frac{S}{R_C}$
- During playout, the buffer is voided in real-time (each second, we consume one second of buffer)
- $B'_{\text{playout}}(t) = \frac{S}{R_C} - 1$: **when $S \leq R_C$ the buffer starts emptying**

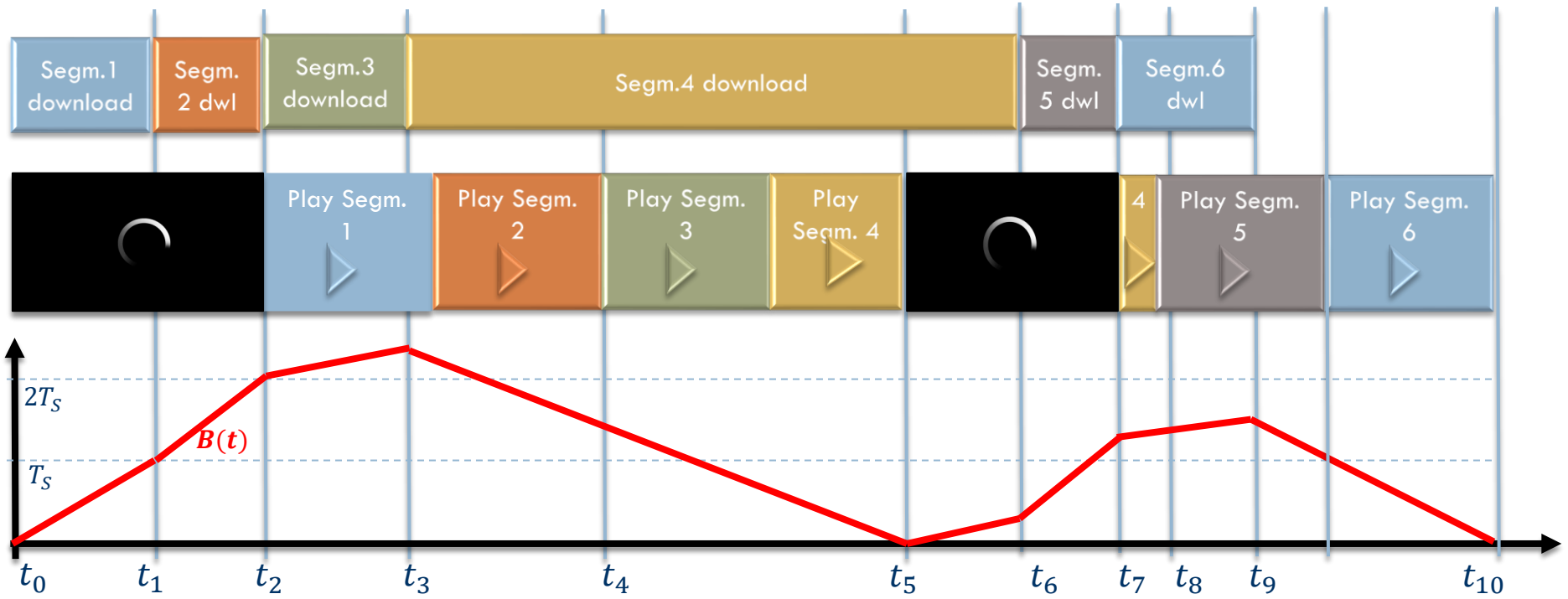


The playout buffer

Between t_0 and t_2 the buffer is filled; between t_1 and t_2 the slope is larger than in the previous interval since the dwl time of the second segment is smaller; during the third interval the buffer keep slowly filling, since again $T_D < T_S$, which implies $R_C < S$

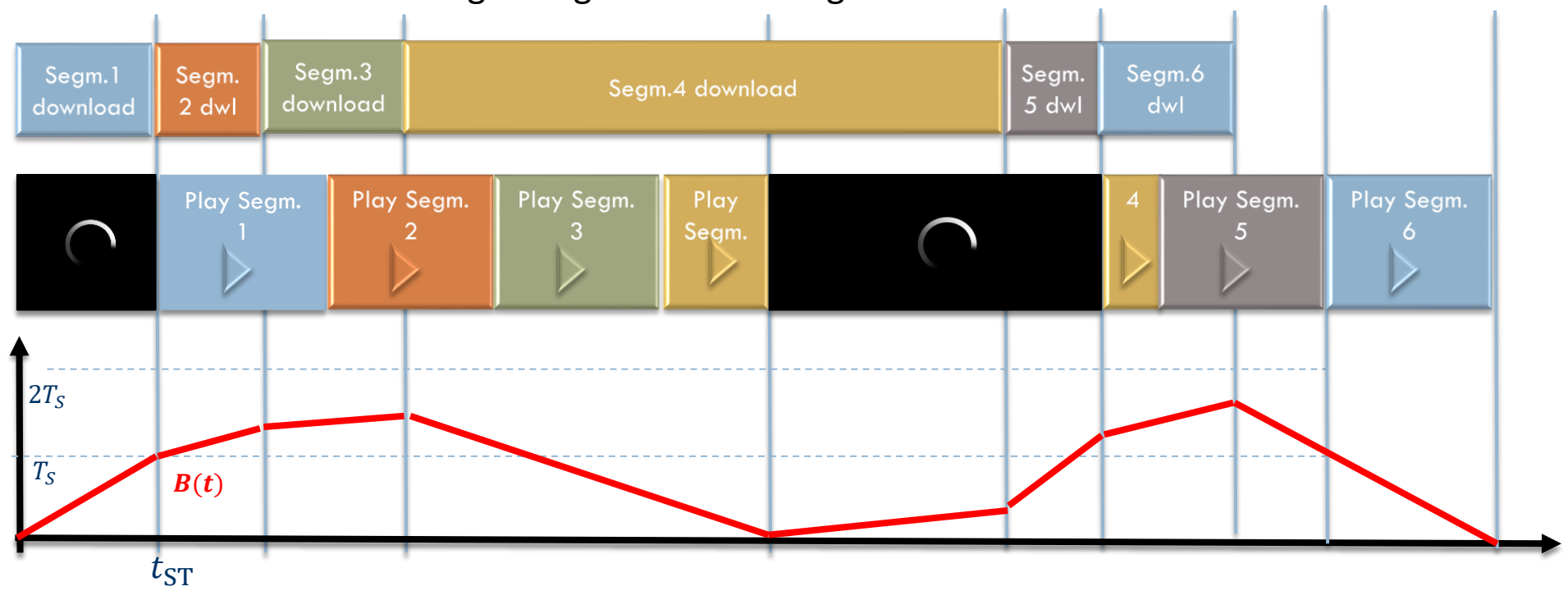
Then, the throughput fall abruptly and $T_D(4)$ is very large: the buffer starts voiding, since $\frac{S}{R_C} < 1$

At t_5 the buffer is empty, and the playout is frozen. The buffer starts refilling slowly until t_6, when the next segment is downloaded at higher speed. At t_7 a full new segment is loaded, and the playout can restart. The next and last two segment are downloaded at $S > R_C$ thus the buffer fills some more. When the last segment is fully received, the buffer is voided with $B'(t) = -1$



The playout buffer

- With the same server and network conditions (i.e., the same download times), let us see what happens when $L = 1$ (i.e., only one segment is downloaded before starting the playout)
- The playout starts quickly but reducing the initial buffer makes it more probable to have long stalling events during the playout
- This is perceived as worse than having a long initial buffering time



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

VIDEO STREAMING QOE

Video Streaming Quality of Experience

- Problem: defining the quality of experience (QoE) for the streaming service

- The global QoE is influenced by:

1. The per-segment video quality
2. The number and duration of re-buffering events
3. The number and amplitude of quality changes
4. The duration of the initial buffering time

Video Streaming Quality of Experience

1. The **per-segment video quality** $Q(n)$

In general, $Q(n) = f\big(R(k_n)\big)$, where i is the number of segment and k_n is the level selected for this segment.
The function f should be a rate-quality function, like $\mathrm{MOS} = f(R)$.
However, we know that the exact behavior of the perceived quality vs. rate relationship largely depends on the content: some images could look perfect at 0.5 bpp while others show visible artifacts at 1 bpp
We also know that the value of the PSNR is only a proxy of the subjective quality
Finally, the perceived quality is, in all relevant cases, a *monotonic increasing function of the coding rate* R
In conclusion, since it is very difficult to model the relationship between MOS and coding rate, it is a common choice to just take a very simple models for f:
$Q(n) = R(k_n)$ or even $Q(n) = k_n$

Video Streaming Quality of Experience

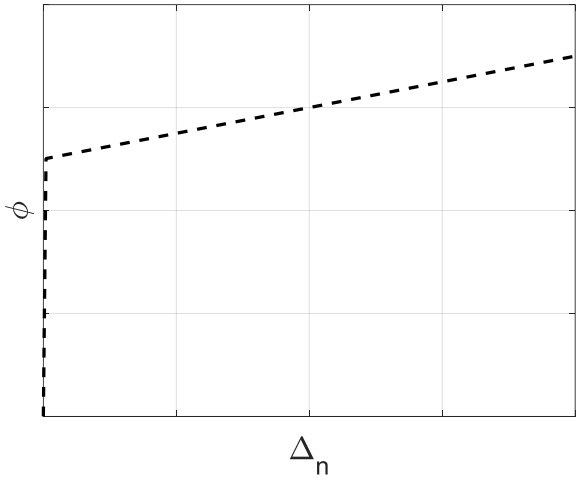
2. The **number and duration** of re-buffering events

Let us refer with Δ_n to the duration of the re-buffering event occurred during the playout of segment number n
If no rebuffering affects segment n, then $\Delta_n = 0$
Since after a rebuffering at least a full segment is downloaded before resuming the playout, there cannot be more than one rebuffering during a single segment playout.
The sheer re-buffering is very disruptive for the QoE, regardless its duration.

In conclusion, the quality of experience during segment n decreases with the duration Δ the rebuffering. The **reduction** of QoE is modeled with a function $\phi(\Delta)$, such that $\phi(0) = 0$, while $\phi(0^+)$ is relatively large but then ϕ increases slowly:

$$\phi(\Delta) = \begin{cases} 0 & \text{if } \Delta = 0 \\ a\Delta + b & \text{if } \Delta > 0 \end{cases}$$

where a is "small" and b is "large"



Video Streaming Quality of Experience

3. The **number** and **amplitude** of **quality changes**
   Any quality variation, both decrease *and increase* are annoying *per se*: it is better to have a constant, maybe somewhat low level of quality than to frequently change the level

4. The **duration** of the **initial buffering** time
   Typically, users allow an initial buffering time if it reduces the number of rebuffering later. However, a too-long initial buffering time also reduces the QoE

A video quality metric for streaming

In conclusion, the **QoE for segment n** can be represented as:

$$J(n) = \lambda_1 k_n - \lambda_2 |k_n - k_{n-1}| - \phi(\Delta_n)$$

The QoE metric $J(n)$ measured over segment n

- increases with the level k_n
- decreases if there is a level change wrt previous segment
- if there is a stall of duration Δ_n , J decreases sharply, i.e. $\phi(0) = 0$, while $\phi(0^+)$ is relatively large

- **The sequence-level QoE** is given by the sum of per-segment QoEs minus the annoyance of the initial buffering:

$$J = \sum_{n=1}^{N} J(n) - \lambda_3 T_{ST}$$

T_{ST} is the time required for downloading L segments with the quality level assigned by the streaming strategy

- It is in general difficult to set the values of λ_i's and the behavior of ϕ. λ_3 is relatively small (good tolerance to initial delay) while ϕ has a discontinuity in 0: even the smallest buffering event is considered annoying

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

ADAPTIVE BITRATE (ABR) STRATEGIES

Adaptive Bit Rate (ABR)

- A client can use a bit rate adaptation (ABR) algorithm to automatically select the “best” quality of the next segment that can be downloaded in time for playback
- ABR is typically non-standardized because is implemented only at the client’s side
- Common ABR algorithms are **throughput-based**, **buffer-based**, and **hybrid** schemes

Throughput-based ABR

- Throughput-based schemes estimate the network throughput and decide on bitrate of next segment to be downloaded
  - Choose frame quality so that estimated download time is not longer than segment playout time
  - E.g.: PANDA, Festive, and Squad

- Z. Li, X. Zhu, J. Gahm, R. Pan, H. Hu, A. C. Begen, and D. Oran, "Probe and adapt: Rate adaptation for http video streaming at scale," IEEE Journal on Selected Areas in Communications, vol. 32, no. 4, 2014.
- J. Jiang, V. Sekar, and H. Zhang, "Improving fairness, efficiency, and stability in http-based adaptive video streaming with FESTIVE," in Proc of Conference on Emerging Networking Experiments and Technologies. Association for Computing Machinery, 2012.

Buffer-based ABR

- Buffer-based algorithms use buffer level to decide on bitrate of next segment
  - Buffer level high/low → frames arrive faster/slower than consumed → quality can be increased/decreased
  - Such algorithms include BBA and **BOLA**

- T.-Y. Huang, R. Johari, N. McKeown, M. Trunnell, and M. Watson, "A buffer-based approach to rate adaptation: Evidence from a large video streaming service," in Proceedings of the 2014 ACM Conference on SIGCOMM. Association for Computing Machinery, 2014.
- K. Spiteri, R. Urgaonkar, and R. K. Sitaraman, "Bola: Near-optimal bitrate adaptation for online videos," in INFOCOM. IEEE, 2016.

Hybrid ABR

- Hybrid algorithms use both throughput prediction and buffer levels
  - ELASTIC, MPC, and ABMA+, DYNAMIC

  - X. Yin, A. Jindal, V. Sekar, and B. Sinopoli, “A control-theoretic approach for dynamic adaptive video streaming over http,” SIGCOMM Comput. Commun. Rev., vol. 45, no. 4, 2015.
  - Beben, Andrzej, et al. "ABMA+ lightweight and efficient algorithm for HTTP adaptive streaming." Proceedings of the 7th International Conference on Multimedia Systems. 2016.

An example of ABR (Adaptive BitRate streaming)

- **The client decides k_n with the goal of maximizing $J = \lambda_1 k_n - \lambda_2 |k_n - k_{n-1}| - \phi(\Delta_n)$**
- We need to decide the quality level for segment n, knowing the previous level k_{n-1}, the playout buffer size $B_0 > 0$ (i.e., B_0 seconds of received video have not been played yet) and an estimation of the throughput $\hat{S}$. The duration of a segment is T_S seconds

Set $J_{\min} = \infty$

For each $\ell = 1 \dots K$, (we test all the possible quality levels: we need to compute the duration of the stall Δ associated to each quality level)

Set $R = R(\ell)$: the coding rate at level k

Set $\beta = \frac{\hat{S}}{R} - 1$: it is the filling rate of the playout buffer

Assuming a constant throughput during the dwl of the segment, the buffer level is $B(t) = B_0 + \beta t$

If $\beta \geq 0$, the buffer will not reach zero during the dwl of segment n: we set $\boldsymbol{\Delta = 0}$ and go to the evaluation of J

If $\beta < 0$, the buffer starts depleting during the download of segment n

…

An example of ABR (Adaptive BitRate streaming)

If $\beta < 0$, the buffer starts depleting during the download of segment n

The buffer reaches zero at time t_0 such that $B_0 + \beta t_0 = 0$, i.e., $\boldsymbol{t_0 = \frac{B_0}{1-\frac{\hat{S}}{R}} = \frac{RB_0}{R-\hat{S}}}$

If $\mathbf{t_0 > T_S}$, the playout of the new segment will finish before the buffer is empty: $\boldsymbol{\Delta = 0}$

else ($\mathbf{t_0 < T_s}$) when the buffer reaches zero, there are $T_R = T_S - \frac{RB_0}{R-\hat{S}}$ secs of the segment to be downloaded yet: there is a stall. The duration of the stall is the time needed to download the rest of the segment plus M new segments

$$\boldsymbol{\Delta = (T_R + MT_S)R/\hat{S}}$$

Evaluate $J = \lambda_1 \ell - \lambda_2 |\ell - k_{n-1}| - \phi(\Delta)$

if $J < J_{\min}$,

set $k_n = \ell$ and $J_{\min} = J$

End for

At the end of the for loop, k_n is the best choice for the quality level (in the current hypotheses)

An example of ABR (Adaptive BitRate streaming)

This algorithm is only one among many possible ABR strategies

It has some simplistic assumptions that might undermine its behavior:

- It assumes a constant throughput during all the download time of the segment
- It also assumes a constant throughout during the rebuffering if a stall happens
- It assumes that the coding rate is constant during the playout, while we know that Intra frames have significantly higher coding rates than other frames
- There is no safeguard level of B: as long as it does not reach zero during the playout of the current segment, no action is taken to prevent stalling

Note that this algorithm only runs on the client side: each client can run a different algorithm, e.g., more «aggressive» (i.e., high quality is demanded, with higher risk of stalls) or more cautious (stall probability is made low, but the average video quality is also low)

Limits of current AS technology

- Risk of freezing/low quality if RTT is large
  - Increase in RTT makes also more difficult to estimate the throughput
- Decisions are entirely client-side
  - Client has only a partial view of the channel conditions
- Adaptation strategy may be suboptimal in the presence of multiple DASH clients or aggressive cross-traffic strategies
- Size/bitrate is not the only aspect impacting perceived quality!

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

MPEG-DASH

MPEG-DASH

- MPEG-DASH is a **standard** for enabling dynamic streaming over HTTP
- DASH is an enabler technology:
  - provides formats to make it possible to have efficient and high-quality streaming over the Internet
- DASH is not:
  - system, protocol, presentation, codec, middleware, client specification

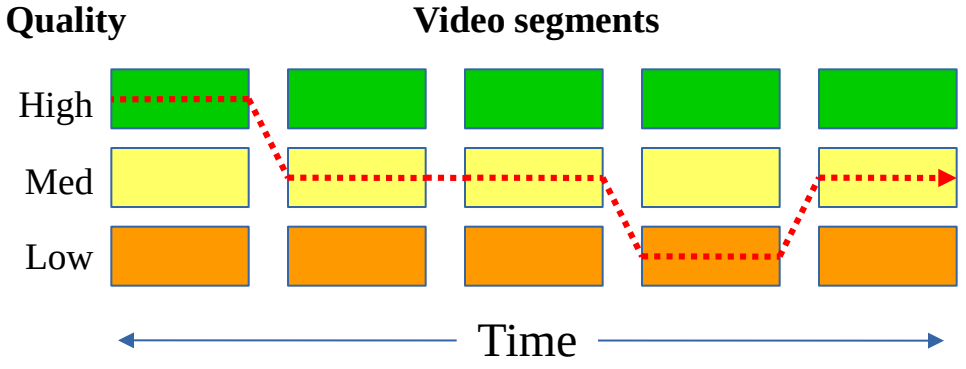
- DASH is one possible solution for streaming:
  - It is open and standardized…
  - But other, proprietary solutions exist

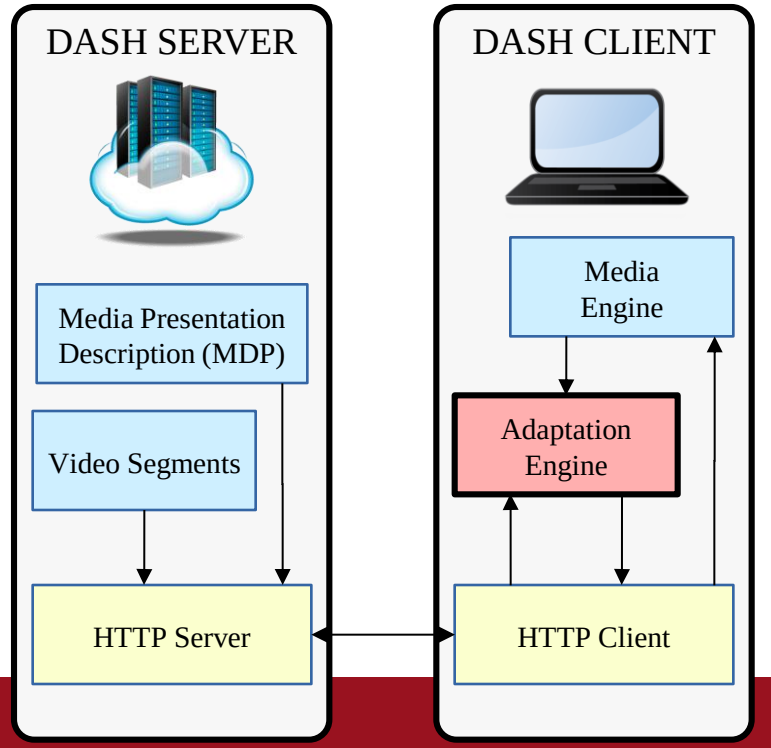
MPEG-DASH Design Principles

- Reuse of existing technologies (containers, codecs, Digital rights management, etc.)
- Deployment on top of HTTP-CDNs
  - HTTP is not filtered by middleboxes
- It targets very high QoE (low start-up, no rebuffering)
- Video quality selection based on network and device capability & user preferences
- Seamless switching among multiple servers
- Move intelligence from network to client
  - enables client differentiation → good for the market
- Enables deployment flexibility (e.g., live, on-demand, time-shift viewing)

DASH - Dynamic Adaptive Streaming over HTTP

- HTTP Standard Server
- Client-side Adaptation Engine
  - The client chooses the proper segments
  - Maximize Quality of Experience (QoE) e.g. avoid rebuffering...





The Media Presentation Description

- The MPD describes segment information (timing, URL, media characteristics)
- Segments only contain media data
- DASH is audio/video codec agnostic
- One or more representations (i.e., versions at different resolutions or bit rates) of multimedia files are available, in order to allow ABR streaming
- However, the **DASH standard does not specify ABR logic**